

Báo cáo: Thiết kế giao tiếp HTTP hiệu năng cao (100.000 TPS)



1. Bối cảnh & Mục tiêu

Thiết kế giao tiếp HTTP giữa Node A và Node B trong hệ thống UDM-like với mục tiêu:

Throughput

100.000 TPS

P95 Latency

< 20ms

P99 Latency

< 50ms

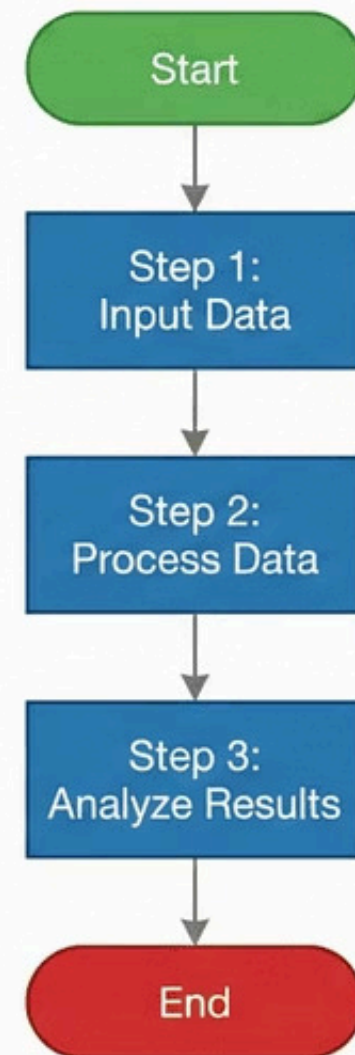
Error Rate

< 0.1%

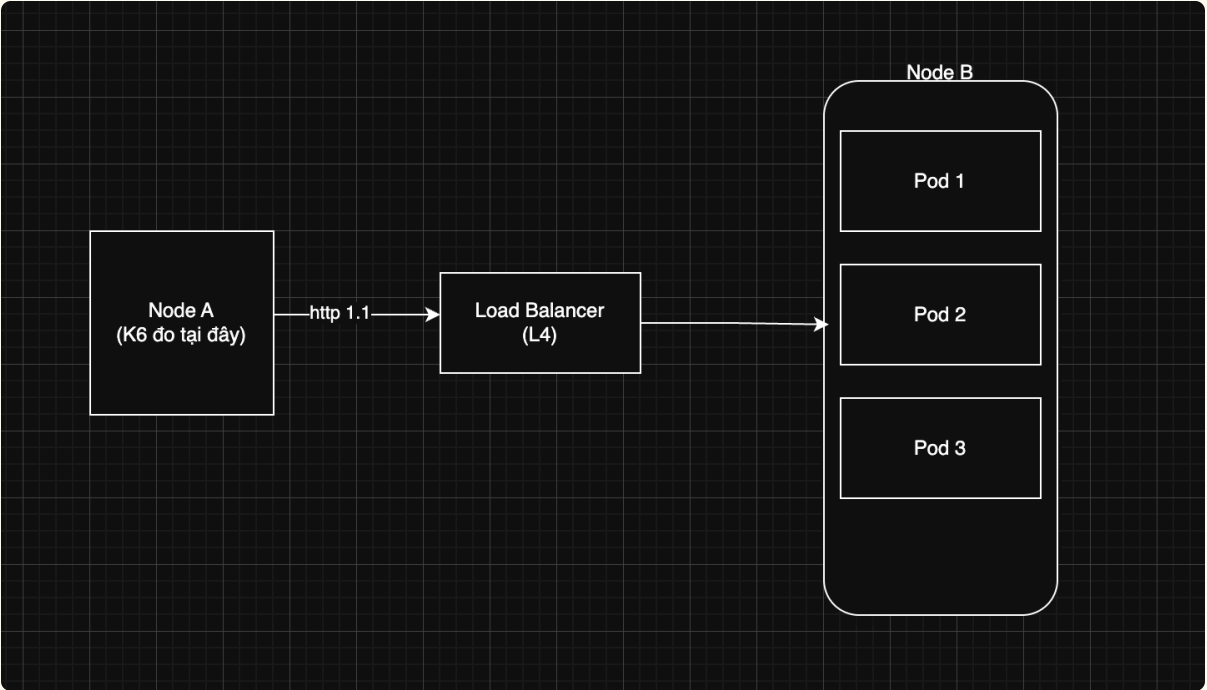


API duy nhất: `GET /subscriber/{supi}` — trả JSON tối giản.

Flowchart: Shows a Process



2. Kiến trúc



Thành phần	Công nghệ	Vai trò
Node A	K6	Load generator
Load Balancer	HAProxy 2.9 — TCP mode (L4)	Phân phối traffic
Node B	Go + fasthttp	API server stateless

📌 **Lý do chọn L4 thay L7:** HAProxy L4 forward TCP thuần, không parse HTTP header → overhead thấp hơn đáng kể ở TPS cao.

3.1 Baseline: net/http

Server Go dùng `net/http` + `json.Marshal` mỗi request.

Target RPS	Actual RPS	P95	P99	Trạng thái
3 000	2 980	15.78 ms	33.04 ms	✓
3 200	3 181	16.66 ms	30.57 ms	✓
3 400	3 361	31.62 ms	49.72 ms	✗
3 600	3 554	39.34 ms	55.29 ms	✗

📌 **Saturation point:** ~3 200 RPS (1 pod, 0.5 CPU)

3.2 Optimized: fasthttp

Hai thay đổi chính:

Thay net/http bằng fasthttp

fasthttp có HTTP parser riêng, dùng object pool cho request/response → tránh allocation và GC pressure ở mỗi request.

Target RPS	Actual RPS	P95	P99	Trạng thái
4 500	4 498	757 μ s	2.32 ms	✓
4 700	4 696	837 μ s	3.06 ms	✓
5 000	4 986	4.09 ms	22.18 ms	✗

📌 **Saturation point:** ~4 700 RPS (1 pod, 0.5 CPU)

3.3 So sánh

Metric	net/http	fasthttp	Cải thiện
Saturation	~3 200 RPS	~4 700 RPS	+43%
P95 tại saturation	~17 ms	< 1 ms	~17x
Alloc/request	4 lần → mem tăng khi chạy 60s với 4500rps thì mem tăng từ 21-40mi	0 vì không có nên mem vẫn giá trị ~21mi	zero-alloc

4. Tối ưu Tầng 2 — Vertical Scaling

Tăng CPU limit của 1 pod để đánh giá hiệu quả scale dọc.

CPU limit	Saturation point	Tỉ lệ
0.5 CPU	~4 700 RPS	1×
1.0 CPU	~9 000 RPS	~2×

2× CPU → ~2× throughput. Không đạt 2× hoàn toàn do overhead cố định của HAProxy và Docker networking.

📌 **Nhận xét:** Vertical scaling có giới hạn vật lý (không thể tăng CPU mãi) và không giải quyết được single point of failure. Horizontal scaling là hướng đúng cho bài toán 100k TPS.

5. Tối ưu Tầng 3 — Horizontal Scaling

Giữ nguyên 0.5 CPU/pod, tăng số pod và đo throughput tổng.

Pods	Saturation point	Tỉ lệ
1 pod	~4 700 RPS	1×
2 pods	~9 000 RPS	~1.9×
3 pods	~13 000 RPS	~2.8×

Throughput tăng gần tuyến tính theo số pod — chứng minh kiến trúc stateless hoạt động đúng: không có shared state giữa các pod, HAProxy phân phối đều.

Mở rộng lên 100.000 TPS

Từ dữ liệu thực đo:

Hiệu năng mỗi pod

Mỗi pod (0.5 CPU, fasthttp) $\approx$ 4 700 RPS

Tính toán số pod

$100\,000\text{ RPS} \div 4\,700\text{ RPS/pod} \approx$ **22 pods**

Với infrastructure thực (Node B cluster riêng, K6 trên máy riêng), 22 pod $\times$ 0.5 CPU = 11 core đủ để đạt 100k TPS trong KPI.

6. Phân tích Bottleneck

6.1 CPU Throttling (Docker cgroup)

Khi container đạt giới hạn CPU (cpus: 0.5)

Container được 50ms CPU / 100ms window → burst request → hết quota sau 50ms → bị freeze 50ms còn lại → latency median tăng lên ~100–140ms

Dấu hiệu nhận biết: median latency ~ 100–140ms, CPU container đứng ở đúng mức giới hạn (50%, 100%...).

Giải pháp: Scale ngang (thêm pod) thay vì tăng CPU limit của 1 pod.

6.2 K6 CPU Saturation

Ở 13.000 RPS, K6 tiêu thụ ~500% CPU (5 core) do JavaScript engine (goja) có overhead cao. Trên môi trường single machine, K6 tranh CPU với Node B làm kết quả benchmark bị nhiễu.

Giải pháp production: Đây là do đang bottleneck ở máy localtest

6.3 HAProxy

HAProxy L4 không phải bottleneck ở mức TPS này. CPU cao (~85%) quan sát được trong benchmark là do chạy chung máy với K6, không phải do HAProxy thiếu capacity.

7. Monitoring

Trong quá trình benchmark, theo dõi 3 chỉ số sau là đủ để phát hiện saturation sớm:

7.1 TCP Connection Count

```
watch -n 1 'ss -tn state established dst :8000  
| wc -l'
```

Tỉ lệ connections/RPS ổn định trong vùng bình thường, tăng đột biến khi hệ thống bão hòa:

7.2 CPU per Pod

```
docker stats --format "table  
{{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
```

Khi CPU container chạm đúng mức giới hạn (cpus limit) và đứng yên ở đó → đang bị cgroup throttle → latency sẽ spike. Scale ngang ngay thay vì chờ error rate tăng.

7.3 K6 Dropped Iterations

dropped_iterations trong K6 output là chỉ số nhanh nhất để phát hiện hệ thống không theo kịp target RPS. Khi dropped tăng đột biến so với các run trước ở cùng RPS → hệ thống đang vào vùng stress.

Config	RPS	Connections	Connections/RPS	Trạng thái
3 pods	9 000	~420	0.047	✓
3 pods	12 000	~525	0.044	✓
3 pods	13 000	~989	0.076	✗

📌 **Ngưỡng cảnh báo:** connections/RPS > 0.06 → cần scale thêm pod.